Project Changes and Remaining Risks

Exoplanet Habitability Catalog

A plain-language record of what the current project copy has changed in response to the identified concerns, what those changes improve, and what still needs work.

What changed at a glance

The nested project copy contains a more organized backend, shared database helpers, a defined model input contract, and a bulk endpoint for identifying score drift. The NASA import and deployment safeguards remain incomplete. The workspace also still contains two project copies, so the team should confirm which copy is the maintained one before treating either as the single source of truth.

Concern	Current change	Status
Mixed route responsibilities	Routes, services, repositories, and predictor now have separate roles.	Improved
Database connections and cursors	Shared pool/transaction helpers manage reuse, rollback, and cursor closing.	Improved
Schema and model contract drift	Canonical names and model feature mapping are documented and centralized in the nested copy.	Improved; verify SQL set together
Per-planet drift requests	A bulk drifted-planets endpoint supports a single catalog query; detail history loads when selected.	Improved
Synchronous NASA import	Still fetches and inserts directly inside the HTTP request.	Open
Security and validation	Input checks exist and debug defaults off, but CORS is unrestricted.	Partial
Duplicate project copy	Both a top-level project and nested copy exist.	Open

Changes explained simply

1. Backend code has clearer jobs

Before, a route could receive a web request, validate its fields, run the model, write SQL, and format the response in one place. The current copy divides those jobs: route files handle HTTP, services validate and coordinate work, repositories contain SQL, and the predictor loads and calls the model.

This makes it easier for a student to find the code responsible for a bug. For example, input rules belong in services.py, while a database query belongs in repositories.py. The route still coordinates the request, so the separation is a practical improvement rather than a fully independent enterprise architecture.

Where to see it: backend/routes/, backend/services.py, backend/repositories.py, backend/predictor.py

2. Database connections have shared management

The shared db.py module obtains connections from a configurable MySQL connection pool. Its transaction helper commits if work succeeds and rolls back if it fails. Query helpers also close cursors in finally blocks, including the stored-procedure helper, and return connections to the pool when finished.

This reduces repeated connection setup and makes cleanup behavior consistent. It does not by itself guarantee that every multi-step business operation is one transaction: current helper calls can each open their own transaction.

Where to see it: backend/db.py and DB_POOL_SIZE in the project setup documentation

3. Database and model field names are documented

The current schema and application use exoplanets.name, while planet_name is not the database column. Prediction history uses predicted_habitability and prediction_timestamp. The model feature contract defines a fixed input order and translates API names such as radius and stellar_flux into the training names expected by the model.

Stellar flux is passed to P_FLUX. Orbital distance is kept as its own value and is not used as a replacement. The saved model pipeline performs its own median imputation, so prediction uses the same saved preprocessing pipeline instead of filling missing features with unrelated API defaults.

The nested copy documents a recommended SQL setup order and includes schema, log trigger, and procedure files that agree on the log field names. Keep that file set together when creating or upgrading a database; do not mix it with the older top-level schema plan without checking names.

Where to see it: docs/architecture-and-data-contract.md, model/feature_contract.py, model/train_model.py, database/schema.sql, database/habitability_log.sql, database/procedures.sql

4. Drift checks can be requested in bulk

The API has a /api/exoplanets/drifted endpoint that returns the planets whose adjacent recorded scores changed by more than a requested threshold. The scientist dashboard can fetch this list once and then mark the matching catalog rows. It requests the detailed score timeline for a planet when that planet is selected.

This changes the expensive catalog-wide pattern from one drift request for every planet to one bulk drift request plus detail requests as needed. The score history itself is still per planet when opened.

Where to see it: backend/routes/exoplanets.py, backend/repositories.py, frontend/js/scientist.js

5. Input checking and app startup are partly improved

Service functions check that names are present, numeric inputs are finite numbers, year-like values are whole numbers, and boolean flags are 0, 1, or empty. Routes return clear client errors for invalid input and missing records, and model-unavailable errors receive a service-unavailable response.

The Flask app enables debug mode only when FLASK_DEBUG is set to 1, so debug is off by default. However, CORS(app) still allows broad cross-origin access. Before deployment, restrict allowed origins, set production configuration deliberately, and review errors and request limits.

Where to see it: backend/services.py, backend/routes/, backend/app.py

6. NASA import and duplicate copies still need decisions

The NASA route currently makes a synchronous request with a 10-second timeout, processes a small top-five query, checks for duplicate planet names, and inserts records one at a time. It does not create or associate host stars, calculate every derived field, or run as a background job. A future version could fetch into a staging area, validate and link records, then import them in a controlled batch.

The repository root also contains an older project alongside the nested project copy. The nested copy has the more complete backend and documentation, but the folder structure alone does not prove which copy the team intends to maintain. Agree on one authoritative project location and move or archive the other only after that decision.

Where to see it: backend/routes/fetch.py, backend/repositories.py; compare the repository root with exoplanet-habitability-catalog-copy/

Suggested next steps

1. Confirm which project copy is authoritative, then keep setup instructions and future code changes in that copy.
2. Run a database setup against the documented schema, log, and procedure files as one matched set; record the database version used by the running app.
3. Limit CORS to the actual frontend address and add production deployment settings before exposing the API beyond a local machine.
4. Plan an asynchronous or staged NASA import that links host stars, validates records, computes required derived values, and reports partial failures.
5. Consider making a single service-level transaction for workflows that write multiple related records, and add tests when the team is ready to verify these behaviors.

How to read the status labels

Improved: The current copy includes a concrete code or documentation change that addresses the concern.

Partial: Some safeguards are present, but an important gap remains.

Open: The concern remains in the current implementation and needs a future change or team decision.